Exponential Information Gathering (EIG): Byzantine Agreement

Curated from Arpit Bhayani's Distributed Systems Architecture Lecture Series • Reference: Pease, Shostak, Lamport (1980)

EXECUTIVE ARCHITECTURAL ABSTRACT

The **Exponential Information Gathering (EIG) Algorithm for Byzantine Agreement** solves distributed consensus in synchronous networks tolerating up to *f* Byzantine (arbitrarily malicious) actors when **n ≥ 3f + 1**. By maintaining an explicit path-attributed attribution tree of depth f + 1 and applying **Recursive Strict Majority Voting** bottom-up, non-faulty nodes systematically eliminate traitor disinformation, proving that Byzantine agreement is achievable in exactly f + 1 rounds.

1. THE BYZANTINE AGREEMENT PROBLEM & THE 3F + 1 IMPOSSIBILITY BOUND

In a synchronous cluster of *n* processes P = {p1, p2, ..., pn}, up to *f* processes are Byzantine (they may lie, contradict themselves, send conflicting values to different peers, or drop messages). Non-faulty processes are loyal. Consensus mandates:

- **Agreement:** All loyal processes must decide upon the identical value v* in V.
- **Validity:** If the Commander (or all loyal nodes) begins with value v, all loyal processes must decide v.
- **Termination:** All loyal processes decide and terminate within f + 1 synchronous rounds.

Why n ≤ 3f is Unsolvable (3 Nodes, 1 Traitor)

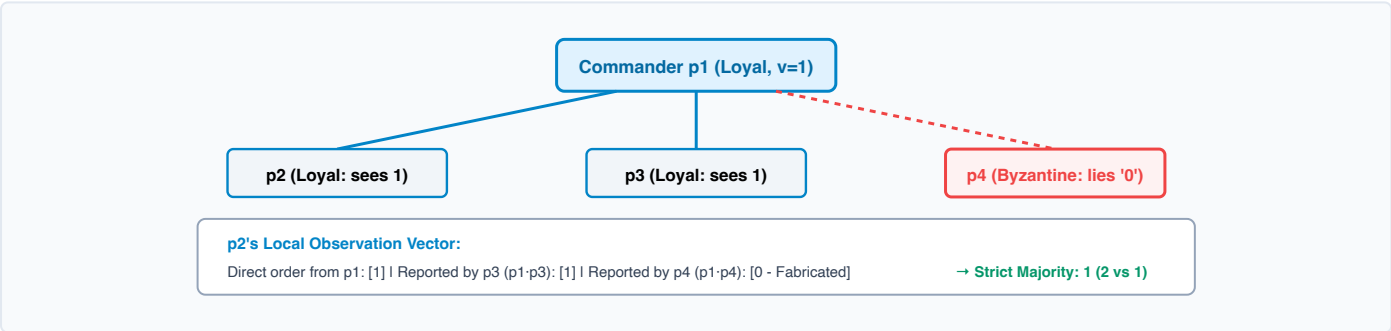
In a 3-node cluster with 1 traitor, Commander C sends 1 to Lieutenant L1 and 0 to L2. When L1 and L2 cross-communicate, L1 receives conflicting claims. L1 cannot distinguish whether the Commander lied (Scenario A) or L2 lied (Scenario B). Under unauthenticated channels, resolving this ambiguity requires at least **n = 3f + 1** nodes so loyal nodes form a strict majority.

The f + 1 Round Latency Bound

Just as in crash failures, defeating cascading Byzantine deceit requires at least **f + 1 rounds** of message exchanges. Over f + 1 rounds, every transmission path of length f + 1 is guaranteed by the Pigeonhole Principle to contain at least one loyal node.

2. BYZANTINE EIG ARCHITECTURE & VECTOR ATTRIBUTION

Each process p maintains a local tree Tp of depth f + 1 where each node represents a transmission path α = j1 j2 ... jk of distinct processes:



3. PROTOCOL EXECUTION & THE RECURSIVE STRICT MAJORITY RULE

The EIG Byzantine agreement algorithm proceeds across two coordinated phases:

Phase 1: Recursive Exchange (f + 1 Rounds)

1. **Round 1:** Each process p broadcasts its initial value vp. Upon receipt, p populates level 1 of its tree: `val_p(j) = received_val`.
2. **Round r (2 ≤ r ≤ f + 1):** Each process broadcasts all node pairs (α, val_p(α)) at level r - 1.
3. Upon receiving message from peer j for path α (where j not in path α), process p records: `val_p(α · j) = reported_val`.

Phase 2: Bottom-Up Majority Reduction

1. **Leaves (level f + 1):**
   `newval_p(α) = val_p(α) if valid else v_default`.
2. **Internal Nodes (level f down to 0):**
   Gather child resolved values:
   `C = { newval_p(α · j) | j not in path(α) }`.
   If a value v has **strict majority** (> |C|/2), set `newval_p(α) = v`; else set `newval_p(α) = v_default`.
3. **Decision:** Final cluster consensus is `v* = newval_p(λ)`.

4. REFERENCE PYTHON IMPLEMENTATION (BYZANTINE SIMULATION)

```
from collections import Counter

class EIGByzantineNode:
    def __init__(self, node_id, n, f, initial_val, is_traitor=False):
        self.id, self.n, self.f = node_id, n, f
        self.initial_val = initial_val
        self.is_traitor = is_traitor
        self.val = {}      # path tuple -> int
        self.newval = {}   # path tuple -> resolved int
        self.default_val = 0

    def round_1_send(self, target_id):
        if self.is_traitor:
            return 1 if target_id % 2 == 0 else 0  # Lie to split votes
        return self.initial_val

    def round_r_send(self, current_round):
        level = current_round - 1
        payload = {p: v for p, v in self.val.items() if len(p) == level}
        if self.is_traitor:
            return {p: (1 - v if v in (0, 1) else v) for p, v in payload.items()}
        return payload

    def resolve_tree(self):
        # 1. Base case: leaf nodes at depth f + 1
        for p, v in self.val.items():
            if len(p) == self.f + 1:
                self.newval[p] = v if v is not None else self.default_val

        # 2. Inductive reduction: level f down to root ()
        for level in range(self.f, -1, -1):
            paths = [p for p in self.val if len(p) == level] if level > 0 else [()]
            for p in paths:
                c_vals = [self.newval[p + (j,)] for j in range(1, self.n+1) if (p + (j,)) in self.newval]
                if not c_vals:
                    self.newval[p] = self.default_val
                    continue
                counts = Counter(c_vals)
                maj_val, top_c = counts.most_common(1)[0]
                self.newval[p] = maj_val if top_c > len(c_vals) // 2 else self.default_val
        return self.newval[()]
```

5. FORMAL CORRECTNESS PROOF: COMMON VALUE & PATH COVERING

The mathematical proof of Byzantine Agreement under n ≥ 3f + 1 relies on backward induction over loyal path labels:

Core Proof Lemmas (Pease, Shostak, Lamport)

Definition (Loyal Path): A node path α = j1 j2 ... jk is *loyal* if the terminal process jk is non-faulty.

Lemma 1 (Common Value on Loyal Nodes): If α is a loyal node, then every loyal process p and q records identical raw values: `val_p(α) = val_q(α)`, and consequently `newval_p(α) = newval_q(α)`.

Lemma 2 (Pigeonhole Covering): Since tree depth is f + 1 and at most f processes are Byzantine, every path from root to leaf contains *at least one loyal node*.

Lemma 3 (Strict Majority Overrides Traitor Lies): For any internal node α at level k ≤ f, the number of child branches is |C(α)| = n - k ≥ (3f + 1) - f = 2f + 1. Since at most f children can be Byzantine, the number of loyal children is at least (2f + 1) - f = f + 1. Because f + 1 > (2f + 1)/2, loyal children form a **strict majority**, ensuring the uncorrupted truth is chosen during bottom-up reduction.

6. PROTOCOL COMPARISON MATRIX

Algorithm	Adversary Model	Resilience Quorum	Communication Rounds	Message Complexity
EIG Byzantine	Arbitrary (Malicious / Byzantine)	n ≥ 3f + 1 (Strict Bound)	f + 1 rounds	O(n^{f+1}) (Exponential)
EIG Crash	Crash-Stop (Fail-Stop)	n ≥ f + 1	f + 1 rounds	O(n^{f+1})
FloodSet	Crash-Stop	n ≥ f + 1	f + 1 rounds	O(n^2) (Polynomial)
PBFT (Castro-Liskov)	Byzantine (Partial Synchrony)	n ≥ 3f + 1	3 phases (Pre-prepare, Prepare, Commit)	O(n^2) per request block

7. SYSTEMS ENGINEERING TAKEAWAYS

1. **Byzantine Tolerance Requires Triple Redundancy:** While crash fault tolerance tolerates f failures with n = 2f + 1 (Paxos/Raft quorums), unauthenticated Byzantine tolerance demands n ≥ 3f + 1. A cluster needing to survive 2 malicious actors must provision at least 7 nodes.
2. **Attribution Masks Dishonesty:** In crash failures, nodes only omit data; in Byzantine failures, nodes invent data. The EIG attribution tree works because it prevents traitors from erasing their footprints—recursive voting attributes messages to senders and forces conflicting reports into the open.
3. **From Theory to Modern Blockchains:** EIG proves that Byzantine agreement is solvable. Practical modern algorithms (Tendermint, HotStuff, PBFT) avoid EIG's O(n^{f+1}) tree explosion by substituting recursive path echoing with cryptographic digital signatures and pipelined view-change quorums.